7. Pipes, Redirection, and Exit Status in Pipelines

07. Pipes, Redirection, and Exit Status in Pipelines

Phase: Phase 3 — Text, pipes, and discovery
Estimated time: 1–2 days
Slug: `07-pipes-redirects`

Learning objectives

- Redirect stdout/stderr to files
- Connect commands with pipes
- Use tee, here-docs, and process substitution ideas
- Understand pipeline exit status (`pipefail`)

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

The Unix philosophy is small tools that do one thing, combined with pipes. Redirection and pipelines let you filter logs, count records, save results and chain programs without writing a single line of a real program. They are also where silent failures hide: a pipeline can "succeed" even when its first stage crashed.

Core concepts

Every process starts with three standard streams, referred to by file descriptor numbers: 0 = stdin (input), 1 = stdout (output), 2 = stderr (errors and diagnostics). Redirection connects them to files; a pipe connects one process's stdout to the next process's stdin.

```
cmd1 --stdout(1)--> | pipe | --stdin(0)--> cmd2 --stdout(1)--> terminal
  \
   +--stderr(2)----------------------------------------> terminal

cmd > out.txt 2> err.txt     (separate files)
cmd > all.txt 2>&1           (stderr follows stdout)
```

Syntax	Effect
`cmd > f`	Write stdout to f, truncating it
`cmd >> f`	Append stdout to f
`cmd < f`	Read stdin from f
`cmd 2> f`	Send stderr to f
`cmd > f 2>&1`	stdout and stderr to f (order matters)
`cmd &> f`	Bash shorthand for the line above
`a | b`	Pipe a's stdout into b
`cmd <<< "text"`	Here-string as stdin
`tee f`	Copy stdin to f and to stdout
`/dev/null`	Discard everything written

Every stage of a pipeline runs at once, as separate processes. By default the exit status of the pipeline is that of the *last* command only. With `set -o pipefail` the status becomes the last non-zero status of any stage, and `${PIPESTATUS[@]}` in bash lists all of them. Scripts commonly start with `set -euo pipefail`.

Tools that work as filters read stdin when given no file: `wc -l`, `sort`, `uniq`, `head`, `tail`, `cut`, `tr`, `grep`.

Worked examples

Redirect and append:

```
echo one > list.txt
echo two >> list.txt
cat list.txt
# sample output: one
# sample output: two
```

Split streams:

```
ls /etc/hostname /nope > out.txt 2> err.txt
cat out.txt
# sample output: /etc/hostname
cat err.txt
# sample output: ls: cannot access '/nope': No such file or directory
```

Classic pipeline, top repeated lines:

```
printf 'b\na\nb\nc\nb\na\n' | sort | uniq -c | sort -rn | head -n 2
# sample output:       3 b
# sample output:       2 a
```

`uniq` only collapses adjacent duplicates, so sort first.

Counting lines from stdin:

```
printf 'x\ny\nz\n' | wc -l
# sample output: 3
wc -l < list.txt
# sample output: 2
```

Using `< file` avoids the file name that `wc -l list.txt` prints.

The pipefail trap:

```
false | cat; echo "status=$?"
# sample output: status=0
set -o pipefail
false | cat; echo "status=$?"
# sample output: status=1
```

Redirect order:

```
ls /nope 2>&1 > /dev/null | wc -l
# sample output: 1
```

Here stderr goes to the pipe (the old stdout) before stdout is sent to /dev/null.

Mini-lab

1. `cd /tmp && mkdir lab7 && cd lab7`.
2. `seq 1 10 > nums.txt` and `wc -l < nums.txt` (prints 10).
3. `grep 5 nums.txt | wc -l` and explain why it prints 1.
4. `ls /etc /nope > o.txt 2> e.txt` and inspect both files.
5. `ls /etc /nope > both.txt 2>&1` then `wc -l both.txt`.
6. `cat nums.txt | tee copy.txt | tail -n 2` to see tee in action.
7. Run `false | true; echo $?` with and without `set -o pipefail`; also print `${PIPESTATUS[@]}`.
8. Write a one-liner that counts unique shells in `/etc/passwd`: `cut -d: -f7 /etc/passwd | sort -u | wc -l`.

Decision guide / failure modes

Use a pipe when the next tool should process data as it streams; use a file redirect when you need to keep the result; use `tee` for both. Use `>>` for logs; use `>` only when overwriting is intended. Use `2>/dev/null` sparingly, since it hides the very errors you need.

Common failures:

- `cmd > f` on a file you also read, e.g. `sort f > f`, empties the file before sort reads it. Use a temp file or `sort -o f f`.
- `cmd 2>&1 > f` instead of `cmd > f 2>&1`: the order is wrong and stderr still hits the terminal.
- Pipeline success hiding failures; use `set -o pipefail`.
- `sudo cmd > /root/file` redirect runs as you; use `| sudo tee`.

- `cat file | grep` is harmless but wasteful; `grep pat file` is simpler.
- `uniq` without `sort` giving wrong counts.

Common mistakes

- Clobbering important files with `>`
- Grepping `ps` for a process and matching the grep itself
- Ignoring stderr while debugging

Exercises

Work in `exercises/07-pipes-redirects/`. Read that folder's `README.md` fully before coding.

```
cd exercises/07-pipes-redirects
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement count_lines(text). It receives the full stdin content as one string and returns the number of lines as an int, behaving like wc -l applied to a stream. A line is counted for each newline-terminated line; but if the text is non-empty and does not end with a newline, the last partial line should still be counted (a friendlier variant than wc, which counts only newline characters). An empty string has 0 lines. A string that is only "" has 1 line. Do not read files or the real stdin; only use the argument. Blank lines count as lines.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/07-pipes-redirects/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What does `2>&1` mean?
2. Why enable `pipefail`?
3. What does `tee` do?

Further reading

- https://www.gnu.org/software/bash/manual/html_node/Redirections.html